

Московский авиационный институт
(национальный исследовательский университет)

Факультет Компьютерные науки и прикладная математика

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Дискретный анализ»

Студент: Савельев Артем

Преподаватель: А. А. Кухтичев

Группа: М8О-206БВ-24

Дата: 5.03.2026

Оценка:

Подпись:

Москва, 2026

Лабораторная работа №1

Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа указанным алгоритмом сортировки за линейное время и вывод отсортированной последовательности.

Вариант сортировки: Поразрядная сортировка (Radix Sort).

Вариант ключа: 64-битные беззнаковые целые числа $(0 \dots 2^{64} - 1)$.

Вариант значения: Строки переменной длины (до 2048 символов), не содержащие символов перевода строки и табуляции.

1 Описание

Требуется написать реализацию алгоритма поразрядной сортировки (Radix Sort) по младшим разрядам (LSD).

Основная идея Radix Sort заключается в последовательной сортировке элементов по разрядам, начиная с младшего и заканчивая старшим. Для стабильной сортировки каждого разряда используется алгоритм сортировки подсчетом (Counting Sort) [1].

Поскольку ключом является 64-битное число (`unsigned long long`), мы рассматриваем его как последовательность из 8 байт (разрядов в системе счисления с основанием 256). Таким образом, алгоритм гарантированно выполняет ровно 8 проходов, что обеспечивает линейное время работы $O(k \cdot n)$, где $k = 8$.

2 Исходный код

Для работы с динамическим массивом был реализован собственный шаблонный класс `Vector`, поддерживающий семантику перемещения (`std::move`), чтобы избежать накладных расходов на копирование длинных строк.

```
#include <iostream>
#include <utility>
#include <string>

template <typename T>
class Vector {
private:
    T* data;
    size_t sz;
    size_t cap;

    void Reserve(size_t new_cap) {
        if (new_cap <= cap) return;
        T* new_data = new T[new_cap];
        for (size_t i = 0; i < sz; ++i) {
            new_data[i] = std::move(data[i]);
        }
        delete[] data;
        data = new_data;
        cap = new_cap;
    }

public:
    Vector() : data(nullptr), sz(0), cap(0) {}

    Vector(size_t n) : data(new T[n]), sz(n), cap(n) {}

    ~Vector() {
        delete[] data;
    }

    Vector(const Vector& other) : data(nullptr), sz(0), cap(0) {
        Reserve(other.cap);
        for (size_t i = 0; i < other.sz; ++i) {
            data[i] = other.data[i];
        }
        sz = other.sz;
    }

    Vector& operator=(Vector other) {
        std::swap(data, other.data);
        std::swap(sz, other.sz);
        std::swap(cap, other.cap);
        return *this;
    }

    void Push_Back(T value) {
        if (sz == cap) {
```

```

        Reserve(cap == 0 ? 8 : cap * 2);
    }
    data[sz++] = std::move(value);
}

T& operator[](size_t i) { return data[i]; }
const T& operator[](size_t i) const { return data[i]; }
size_t size() const { return sz; }

T* begin() { return data; }
T* end() { return data + sz; }
const T* begin() const { return data; }
const T* end() const { return data + sz; }
};

struct Item {
    unsigned long long key;
    std::string value;

    Item() : key(0) {}
    Item(unsigned long long k, std::string v) : key(k), value(std::
        move(v)) {}
};

unsigned char GetByte(unsigned long long key, int byteNum) {
    return (key >> (byteNum * 8)) & 0xFF;
}

void RadixSort(Vector<Item>& a) {
    size_t n = a.size();
    if (n <= 1) return;

    Vector<Item> b(n);

    for (int byteNum = 0; byteNum < 8; ++byteNum) {
        size_t count[256] = {0};

        for (size_t i = 0; i < n; ++i) {
            count[GetByte(a[i].key, byteNum)]++;
        }

        for (int i = 1; i < 256; ++i) {
            count[i] += count[i - 1];
        }

        for (size_t i = n; i > 0; --i) {
            unsigned char byte = GetByte(a[i - 1].key, byteNum);
            b[--count[byte]] = std::move(a[i - 1]);
        }
    }
}

```

```

        for (size_t i = 0; i < n; ++i) {
            a[i] = std::move(b[i]);
        }
    }
}

int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(0);

    Vector<Item> items;
    unsigned long long key;
    std::string val;

    while (std::cin >> key) {
        if (std::cin.get() == '\t') {
            std::getline(std::cin, val);
            items.Push_Back(Item(key, std::move(val)));
        }
    }

    RadixSort(items);

    for (size_t i = 0; i < items.size(); ++i) {
        std::cout << items[i].key << "\t" << items[i].value << "\n";
    }

    return 0;
}

```

3 Тест производительности

Тест производительности сравнивает время работы реализованного алгоритма поразрядной сортировки (Radix Sort) с библиотечной функцией стабильной сортировки `std::stable_sort` из STL C++.

Данные для теста (100 000 строк) были сгенерированы отдельным Python-скриптом. Время измерялось в микросекундах (us) с использованием библиотеки `<chrono>`.

```
artems@MacBook-Air-artem-2 lab1 % make run-bench
g++ -std=c++14 -O3 -pedantic -Wall -Wextra src/benchmark.cpp src/sort
.cpp -o benchmark_run
python3 src/generator.py 100000 input.txt
./benchmark_run < input.txt
Count of lines: 100000
-----
Radix Sort (LSD):   3506 us
STL Stable Sort:   7268 us
-----
Speedup: Radix is 2.07302x faster!
```

Как видно из результатов бенчмарка, реализованная поразрядная сортировка обрабатывает примерно в 2 раза быстрее стандартной сортировки C++ на объеме данных в 100 000 элементов. Это подтверждает линейную асимптотику $O(n)$ алгоритма Radix Sort по сравнению с $O(n \log n)$ у `std::stable_sort`.

4 Выводы

Выполнив первую лабораторную работу по курсу «Дискретный анализ», я изучил принципы работы линейных алгоритмов сортировки и реализовал стабильный алгоритм поразрядной сортировки (Radix Sort LSD) в связке с сортировкой подсчетом.

В процессе выполнения лабораторной работы я:

- Углубил знания языка C++, реализовав собственный контейнер динамического массива (`Vector`), аналогичный `std::vector`.
- Научился применять семантику перемещения (`std::move`) для оптимизации копирования тяжелых объектов (строк), что критически важно для производительности сортировки.
- Разобрался с нюансами быстрого парсинга данных и оптимизацией стандартных потоков ввода-вывода (`std::cin.tie`, `std::ios_base::sync_with_stdio`).
- Написал Python-скрипты для генерации тестов и автоматической проверки (чекер) правильности сортировки и целостности данных.
- Освоил систему автоматизации сборки `make`.

Результаты бенчмарков наглядно показали преимущество специализированных линейных алгоритмов над универсальными сортировками сравнением при больших объемах данных.

Список литературы

- [1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 3-е издание*. — Издательский дом «Вильямс», 2013. — 1328 с.
- [2] *C++ reference*.
URL: <https://en.cppreference.com/>